

Comprehensive Project Report: Legal RAG Chatbot

1. Abstract

The rapid advancement of **Large Language Models (LLMs)** has transformed natural language processing. However, deploying these models in high-stakes domains such as the legal sector presents significant risks, primarily due to "hallucinations"—the generation of fluent but factually incorrect information. This project introduces a **Legal Retrieval-Augmented Generation (RAG) Chatbot** designed to overcome these limitations. By integrating a sophisticated document retrieval pipeline with the reasoning capabilities of **Google's Gemini 2.5 Flash** model, this system ensures that all generated answers are strictly grounded in user-provided legal reference documents. The architecture leverages **PyPDF** for text extraction, **FAISS** for efficient vector similarity search, and Streamlit for an interactive user interface. This report provides an exhaustive analysis of the project's architecture, implementation details, challenges faced, and future scope.

2. Introduction

2.1 Problem Statement

Legal professionals, researchers, and clients frequently deal with massive, dense documents such as contracts, case laws, and compliance manuals. Finding specific information within these documents using traditional keyword search is inefficient, as it lacks semantic understanding. Conversely, asking a general-purpose LLM to interpret legal concepts is dangerous because the model lacks access to the specific private documents and may confidently invent plausible-sounding legal precedents.

2.2 Objectives

The primary objective of this project is to build a reliable, document-grounded conversational agent. Specific goals include:

- **Zero Hallucination Tolerance:** Forcing the LLM to answer only using the provided text.
- **Semantic Search Capability:** Utilizing vector embeddings to understand the "meaning" of a query rather than just matching keywords.
- **Source Transparency:** Providing exact page numbers and excerpts for every answer generated to build user trust.
- **Scalability and Modularity:** Designing the system so that the embedding models, vector database, and LLM can be independently upgraded.

3. Background Technologies

3.1 Large Language Models (LLMs) and Gemini

LLMs are deep learning models trained on vast amounts of text data. For this project, **Google's Gemini 2.5 Flash** is utilized for the generation phase. Gemini 2.5 Flash provides an optimal balance of high-speed inference, cost-effectiveness, and strong reasoning capabilities.

3.2 Retrieval-Augmented Generation (RAG)

RAG is a hybrid architecture that marries the generative power of LLMs with a dedicated retrieval mechanism. Instead of relying on the model's internal parametric memory (which is static and prone to hallucinations), RAG dynamically fetches relevant information from an external database at runtime. This external context is injected into the LLM's prompt, effectively giving the model an "open book" to read from before answering.

3.3 Vector Embeddings and FAISS

To achieve semantic search, text must be converted into high-dimensional numerical vectors (embeddings). The text-embedding-004 model is used to

generate these representations. Facebook AI Similarity Search (FAISS) is employed as the vector database. FAISS is highly optimized for fast, exact, or approximate nearest-neighbor search in dense vector spaces.

4. System Architecture

The Legal RAG Chatbot is divided into two distinct pipelines: **Data Ingestion** and **Inference**.

4.1 The Ingestion Pipeline (Offline Phase)

1. **Document Loading:** The system ingests a legal PDF (`data/legal_reference.pdf`).
2. **Parsing and Cleaning:** Raw text is extracted page-by-page. Formatting inconsistencies are normalized.
3. **Chunking:** The text is split into overlapping chunks (e.g., 1000 characters with a 150-character overlap). This prevents legal clauses from being severed mid-sentence.
4. **Vectorization:** Each chunk is passed to the embedding API.
5. **Indexing:** The resulting vectors are **L2-normalized** and stored in a FAISS Flat Inner Product index, alongside a JSON file tracking the original text and page numbers.

4.2 The Inference Pipeline (Runtime Phase)

1. **Query Processing:** The user's natural language question is embedded using the same `text-embedding-004` model.
2. **Similarity Search:** The query vector is compared against the FAISS index using **cosine similarity** to retrieve the top-K most relevant chunks.
3. **Prompt Construction:** A stringent system prompt is combined with the retrieved chunks and the user's query.
4. **Generation:** Gemini processes this augmented prompt and streams the response back to the Streamlit UI, alongside the citation metadata.

5. Detailed Implementation & Code Structure

5.1 Document Parsing (`pdf_loader.py`)

This module utilizes the `pypdf` library. Unlike simple extractors, it maintains a mapping of text to its corresponding `page_number`. This is critical for the legal domain where citing the exact page of a contract is mandatory. A light cleanup function normalizes whitespace without mangling the legal formatting.

5.2 Context-Aware Chunking (`chunking.py`)

The chunking algorithm employs a sliding window approach. By setting a `chunk_size` of 1000 and `chunk_overlap` of 150, the system ensures that context flows seamlessly between chunks. The algorithm attempts to break chunks at natural paragraph or sentence boundaries (looking for `\n\n` or `.`) rather than cutting words in half.

5.3 Embedding Management (`embeddings.py`)

This module handles the interaction with the Gemini Embedding API. To optimize network usage and cost, texts are batched (up to 100 per request). The module also implements **exponential backoff and retry logic** (`MAX_RETRIES = 3`) to gracefully handle transient API rate limits. All vectors are explicitly L2-normalized to ensure that FAISS inner product calculations equate to cosine similarity.

5.4 Vector Database (`vectorstore.py`)

Instead of relying on heavy, hosted vector databases, this project utilizes a lightweight, local FAISS index (`IndexFlatIP`). This choice is highly intentional: for single legal documents, an **exact-search flat index** is incredibly fast and removes the need for complex infrastructure.

5.5 RAG Orchestration (rag_chain.py)

This is the core reasoning engine. The `SYSTEM_PROMPT` is meticulously engineered to enforce strict boundaries. It commands the model to: "Base your answer strictly on the provided excerpts. Do not use outside knowledge." It also manages conversational history, translating Streamlit's message format into Gemini's `Content` and `Part` schema.

5.6 User Interface (app.py)

Streamlit provides a reactive, modern chat interface. The UI features:

- A sidebar for **API key management** and hyperparameter tuning (e.g., adjusting the `TOP_K` retrieval count).
- **Session state management** to preserve the chat history.
- An expandable "**Sources**" accordion that dynamically renders the exact excerpts and similarity scores used for every generated answer.

6. Challenges and Mitigation Strategies

6.1 LLM Hallucinations and Prompt Drift

- **Challenge:** Initially, the LLM would occasionally supplement answers with its pre-trained legal knowledge, which could contradict the specific uploaded contract.
- **Mitigation:** The system prompt was fortified with explicit negative constraints. Furthermore, a **low temperature setting** (`temperature=0.2`) was enforced during generation to prioritize deterministic, factual outputs over creative ones.

6.2 Context-Aware Text Chunking

- **Challenge:** Hard character cutoffs often resulted in a retrieved chunk starting mid-sentence, confusing the LLM.

- **Mitigation:** The `chunking.py` script was upgraded to search backward for the nearest period or newline character before making a split, preserving semantic integrity.

6.3 Secure API Key Management

- **Challenge:** Ensuring that API keys were not hardcoded or exposed in version control.
- **Mitigation:** Implemented a multi-tier resolution strategy: Streamlit Secrets, local `.env` variables, and a password-masked sidebar input.

7. Setup, Deployment, and User Guide

7.1 Local Development

Users must clone the repository, initialize a Python virtual environment, and install dependencies from `requirements.txt` (which includes `streamlit`, `google-genai`, `faiss-cpu`, and `pypdf`). After placing their target document in the `data/` folder, the user runs `python ingest.py` to build the FAISS index. Finally, `streamlit run app.py` launches the local server.

7.2 Cloud Deployment

The application is designed to be seamlessly deployed on Streamlit Community Cloud. By committing the codebase to GitHub and securely adding the `GEMINI_API_KEY` to the Streamlit dashboard's Secrets manager, the application can be hosted globally for free.

8. Conclusion and Future Scope

The Legal RAG Chatbot successfully demonstrates how generative AI can be safely and effectively deployed in domains requiring strict factual adherence. By isolating the knowledge base from the LLM's parametric memory, the system provides accurate, verifiable, and transparent answers.

8.1 Future Scope

- **Multi-Document Support:** Upgrading the ingestion pipeline to handle entire directories of legal documents.
 - **OCR Integration:** Integrating tools like Tesseract or Google Cloud Vision to process scanned, image-only PDFs.
 - **Advanced Retrieval Techniques:** Implementing **Hybrid Search** (combining FAISS vector search with BM25 keyword search) and Re-ranking models.
 - **User Authentication & Cloud Storage:** Moving from local FAISS files to a managed vector database (like Milvus or Qdrant).
-

Report Prepared By: Harsh Giri

Review Date: ☞ Date